APPLICATION

FUNCTIONALITY MONITORING

Ensuring business logic and service interactions behave as expected

Enterprise Observability Series

AGENDA

01
Introduction & Objective

What is application functionality monitoring and why it matters

02
Application Flow Monitoring

Business logic, workflow tracking, and incident detection

03
API Response Monitoring

Latency, failure rates, anomaly detection

04
How to Monitor

Tools, alert conditions, deviation analysis

05
Monitoring Tools

Dynatrace, Amazon CloudWatch, Splunk

06
Key Takeaways

Summary and recommendations

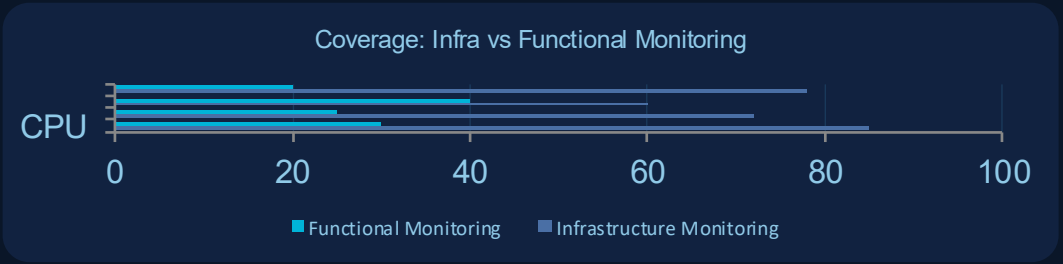
Introduction & Objective

What is Application Functionality Monitoring?

Application functionality monitoring focuses on ensuring that the business logic and service interactions of an application behave as expected.

While infrastructure monitoring tracks system health (CPU, memory, network), functional monitoring verifies the application actually performs its intended business operations correctly.

This is critical because many production incidents occur even when infrastructure metrics appear healthy.



OBJECTIVE

Ensure business logic and service interactions perform correctly across all workflows, APIs, and user-facing operations and detecting failures before they impact customers.

Two Key Monitoring Metrics

Application Flow

End-to-end workflow execution

API Response

Performance & reliability metrics

Application Flow Monitoring

What It Measures

Application flow monitoring measures the end-to-end execution of business transactions or workflows, validating that the sequence of steps required completes correctly.

Start → **Auth** → **Process** → **DB Write** → **Notify** → **Complete**

Typical Business Workflow Steps

User Login Flow

Auth → Session creation → Profile load

Order Checkout Process

Cart → Payment → Inventory → Confirm

Payment Processing

Gateway → Authorization → Settlement

Data Submission Pipeline

Validate → Transform → Store → Notify

Application Flow — Why It Matters

Infrastructure health ≠ Application health

Scenario	Infrastructure Health	Application Flow
Payment gateway failure	✓ Servers healthy	✗ Checkout fails
Logic bug in deployment	✓ CPU/memory normal	✗ Order processing stops
Database schema mismatch	✓ DB running	✗ API transactions fail

Flow Monitoring Detects:

- **Broken Business Logic**
- **Partial Transaction Fails**
- **Dependency Failures**
- **Deployment Regressions**
- **Misconfigurations**

Application Flow — Alert Handling & Error Codes

Alert Triggers

- Workflow success rate drops below threshold
- Specific workflow step fails repeatedly
- Transaction latency exceeds baseline
- Critical business transaction fails

Error Code Severity Mapping

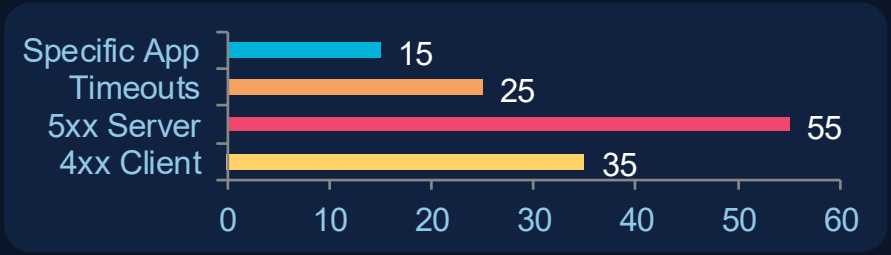
Error Pattern	Severity
Minor increase in 4xx	Low
Sustained 5xx errors	High
Critical dependency error	Critical
Auth failures across services	High

EXAMPLE ALERT

Alert: Checkout Success Rate < 95% for 5 minutes | Severity: Critical | Impact: Customers cannot complete purchases

Operations Team Actions:

1. Identify which business capability is broken
2. Determine which service/dependency failed
3. Escalate to appropriate service owner



API Response Monitoring

Performance, Reliability & Health of Service Endpoints

What It Measures

- Response latency
- Success vs failure status
- Throughput
- Error responses
- Timeouts

Evaluates Endpoints

- Respond correctly
- Respond within acceptable time
- Maintain reliability under load

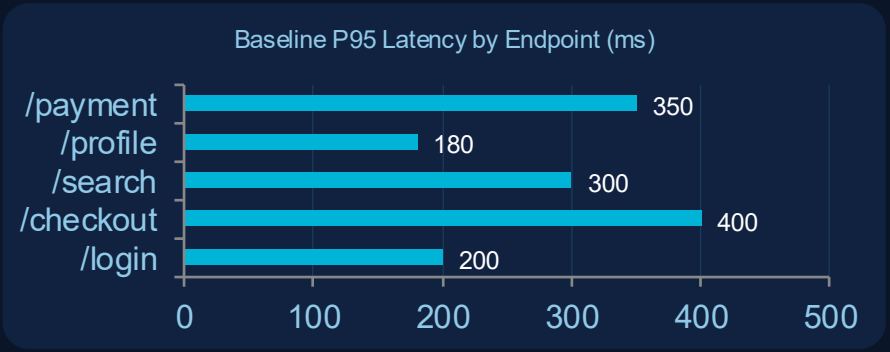
APIs Power:

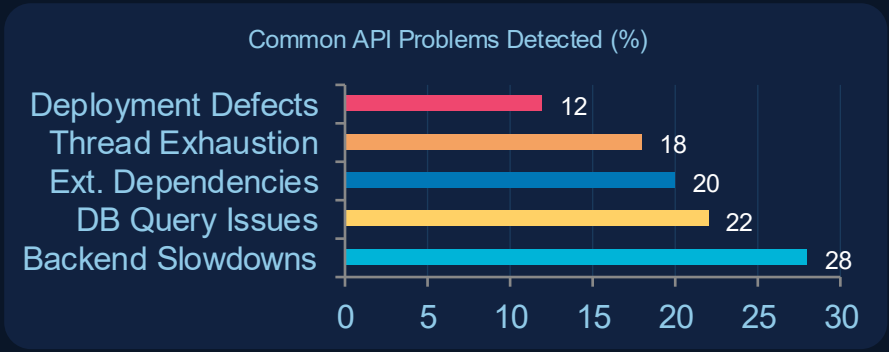
- Web Apps
- Mobile Apps
- Microservices
- 3rd-Party Integrations

Cascade Risk

API failures cascade quickly across distributed systems, impacting multiple downstream services simultaneously.

A single failing endpoint can bring down web apps, mobile clients, microservices, and third-party integrations at once.



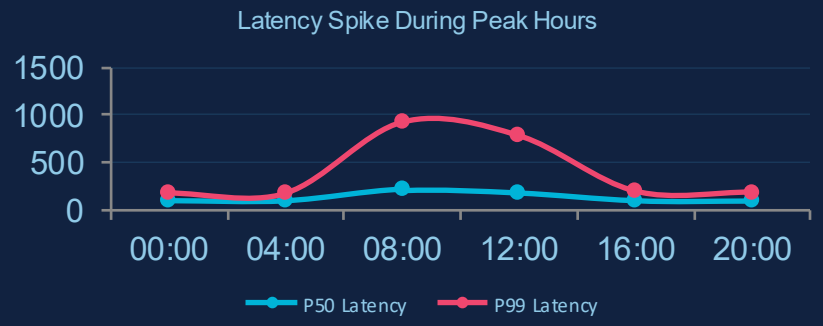


API Response — Latency & Failure Rate

Response Time / Latency

P95	P99
320ms	900ms

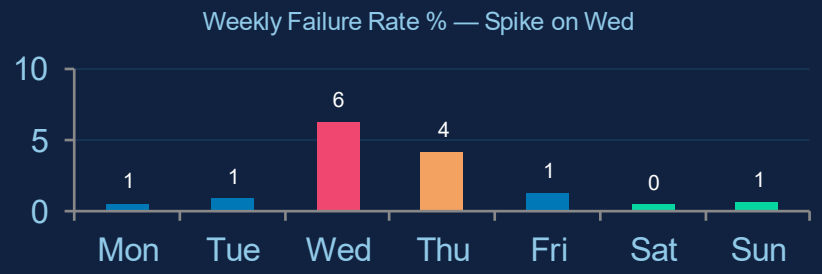
Percentile monitoring reveals tail latency issues hidden by averages.



Failure Rate Monitoring

Failure Rate = Failed Requests / Total Requests

Failure Rate	Interpretation
< 1%	Normal
1–3%	Warning
> 5%	Critical
> 10%	Major Outage



API Response — Anomaly Detection

Identifying unexpected deviations without relying solely on static thresholds

Techniques Used

Statistical Baselines

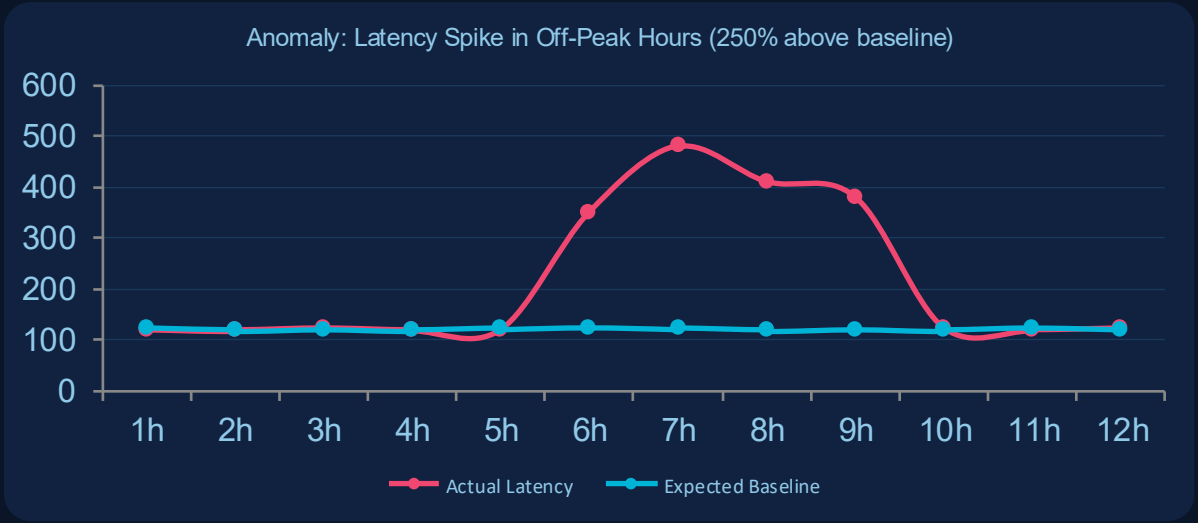
Standard deviation, seasonal baselines, moving averages

Machine Learning

Multi-metric anomaly detection, hidden correlation analysis, early degradation signals

Trend Analysis

Gradual degradation detection before total failure occurs



Anomaly Escalation Process:

1. Flag unusual behavior
2. Correlate across services
3. Generate alert for investigation
4. Incident declared & mitigated

Monitoring Tools

Industry-leading platforms for application functionality monitoring

Dynatrace

AI-driven Observability

- Automatic instrumentation — no manual setup
- PurePath distributed tracing
- Davis AI engine — root cause analysis
- Smartscape topology mapping
- Automatic health scoring

Amazon CloudWatch

Native AWS Monitoring

- Native metrics, logs, events for AWS
- Alarm-driven automation & scaling
- EC2, ECS, Lambda, RDS monitoring
- SNS notifications & Lambda triggers
- Auto-remediation integration

Splunk

Observability & Analytics

- Log monitoring & error detection
- Metrics tracking (latency, error rates)
- Distributed tracing / APM
- Real-time dashboards
- ML-based anomaly detection

Dynatrace — AI-Driven Observability

Performance Monitoring

Auto-detects slow transactions, service latency, code-level bottlenecks & DB query delays via Smartscape topology.

PurePath Distributed Tracing

Full request path across services: User → API Gateway → Order Service → Payment Service → Database.

Health Monitoring

Automatic health scores based on error rate, latency & resource saturation. Triggers 'Problem' incidents.

Davis AI Engine

Correlates events, suppresses alert storms, identifies root cause automatically, reduces noise & false positives.

PurePath Trace Example

Auto-Problem Detection

Davis AI auto-creates a 'Problem' incident when health degrades, linking root cause to the slow Payment Service.

Amazon CloudWatch & Splunk

Amazon CloudWatch

Native AWS Infrastructure & App Monitoring

- ✓ Collects metrics, logs, and events from AWS resources
- ✓ EC2 CPU, memory, network, disk I/O monitoring
- ✓ Load balancer health tracking
- ✓ Alarm-triggered SNS notifications
- ✓ Lambda automation on threshold breach
- ✓ Auto-scaling EC2 on high CPU

CloudWatch alarm: EC2 CPUUtilization > 85% for 5min → SNS → Lambda auto-scales EC2

Splunk

Observability, Analytics & SRE Platform

Log Monitoring

Collects & analyzes application logs to detect errors, exceptions, and abnormal behavior

Metrics Monitoring

API response time, error rates, request throughput tracking

Distributed Tracing/APM

Follows transactions across services, identifies latency & failing components

Real-Time Dashboards

Visual insights into application health, service dependencies & business transactions

Splunk reduces MTTR by enabling teams to detect, analyze, and respond to incidents faster via log + metric correlation.

Key Takeaways

Application Functionality Monitoring — Summary

01 Functional ≠ Infrastructure

Healthy servers don't mean healthy apps. Functional monitoring is essential to catch logic bugs, schema mismatches, and workflow failures invisible to infra monitoring.

02 Application Flow Is Business-Critical

End-to-end workflow monitoring directly maps to user-facing operations. A broken checkout or payment flow has immediate revenue and customer impact.

03 API Monitoring = First Line of Defense

APIs are the primary interaction layer. Monitoring latency, failure rates, and anomalies enables early detection before cascading failures occur.

04 Anomaly Detection Finds Unknown Failures

Static thresholds miss gradual degradation. ML-based anomaly detection identifies subtle, previously unknown failure patterns across distributed services.

05 Right Tools Accelerate Response

Dynatrace (AI root cause), CloudWatch (AWS native), and Splunk (log+metric analytics) together form a comprehensive observability stack to minimize MTTR.

06 Treat Functional Alerts as High Severity

Application flow and API failures directly impact customers. They should be treated as high-severity incidents with immediate escalation and mitigation.